Clases (POO) en Python

Sitio: CAMPUS
Curso: Python - 2º DAW
Libro: Clases (POO) en Python
Imprimido por: MARÍA HAIZHEUSKAYA
Día: jueves, 29 de octubre de 2026, 21:46

Tabla de contenidos

1. Clases y ámbitos
2. Objetos, Instancias y Métodos
3. Ejemplo POO Teletubbies
4. Herencia
    4.1. Herencia Múltiple y Polimorfismo
    4.2. MRO en Python
    4.3. Herencia múltiple cooperativa
5. Variable privadas y Tipado dinámico
6. Iteradores y Generadores

1. Clases y ámbitos

Clases

Las clases proveen una forma de empaquetar datos y funcionalidad juntos. Al crear una nueva clase, se crea un nuevo tipo de objeto, permitiendo crear nuevas instancias de ese tipo. Cada instancia de clase puede tener atributos adjuntos para mantener su estado. Las instancias de clase también pueden tener métodos (definidos por su clase) para modificar su estado.

Comparado con otros lenguajes de programación, el mecanismo de clases de Python agrega clases con un mínimo de nuevas sintaxis y semánticas. Es una mezcla de los mecanismos de clases encontrados en C++ y Modula-3. Las clases de Python proveen todas las características normales de la Programación Orientada a Objetos: el mecanismo de la herencia de clases permite múltiples clases base, una clase derivada puede sobreescribir cualquier método de su(s) clase(s) base, y un método puede llamar al método de la clase base con el mismo nombre. Los objetos pueden tener una cantidad arbitraria de datos de cualquier tipo. Igual que con los módulos, las clases participan de la naturaleza dinámica de Python: se crean en tiempo de ejecución, y pueden modificarse luego de la creación.

En terminología de C++, normalmente los miembros de las clases (incluyendo los miembros de datos), son públicos, y todas las funciones miembro son virtuales. Como en Modula-3, no hay atajos para hacer referencia a los miembros del objeto desde sus métodos: la función método se declara con un primer argumento explícito que representa al objeto, el cual se provee implícitamente por la llamada. Como en Smalltalk, las clases mismas son objetos. Esto provee una semántica para importar y renombrar. A diferencia de C++ y Modula-3, los tipos de datos integrados pueden usarse como clases base para que el usuario los extienda. También, como en C++ pero a diferencia de Modula-3, la mayoría de los operadores integrados con sintaxis especial (operadores aritméticos, de sub-índice, etc.) pueden volver a ser definidos por instancias de la clase.

Unas palabras sobre nombres y objetos

Los objetos tienen individualidad, y múltiples nombres (en muchos ámbitos) pueden vincularse al mismo objeto. Esto se conoce como *aliasing* en otros lenguajes. Normalmente no se aprecia esto a primera vista en Python, y puede ignorarse sin problemas cuando se maneja tipos básicos inmutables (números, cadenas, tuplas). Sin embargo, el aliasing tiene un efecto posiblemente sorpresivo sobre la semántica de código Python que involucra objetos mutables como listas, diccionarios, y la mayoría de otros tipos. Esto se usa normalmente para beneficio del programa, ya que los renombres funcionan como punteros en algunos aspectos.

Ámbitos y espacios de nombres en Python

Antes de ver clases, primero debo decirte algo acerca de las reglas de ámbito de Python. Las definiciones de clases hacen unos lindos trucos con los espacios de nombres, y necesitás saber cómo funcionan los alcances y espacios de nombres para entender por completo cómo es la cosa.

Definiciones Clave

- **Espacio de nombres:** Es una relación de nombres a objetos. Muchos están implementados como diccionarios de Python. Ejemplos: nombres incluidos (builtins), nombres globales en un módulo, y nombres locales en una función. No hay relación entre los nombres de espacios de nombres distintos.
- **Atributo:** Cualquier cosa después de un punto (ej. en `z.real`, *real* es un atributo del objeto *z*). Pueden ser de solo lectura o de escritura. Los modificables se pueden eliminar con la declaración `del`.
- **Ámbito:** Es una región textual de un programa en Python donde un espacio de nombres es accesible directamente.

Los espacios de nombres se crean en diferentes momentos y con diferentes tiempos de vida:

- El que contiene los **nombres incluidos** se crea al iniciar el intérprete y nunca se borra (`builtins`).
- El **espacio de nombres global** de un módulo se crea cuando se lee su definición y dura hasta que el intérprete finaliza (ej. módulo `__main__`).
- El **espacio de nombres local** a una función se crea cuando es llamada y se elimina al retornar o lanzar una excepción.

Resolución de Ámbitos

En cualquier momento durante la ejecución, hay 3 o 4 ámbitos anidados cuyos espacios de nombres son directamente accesibles:

1. El alcance más interno, inspeccionado primero (nombres locales).
2. Los alcances de cualquier función que encierra a otra (nombres no locales, pero no globales).
3. El penúltimo alcance (nombres globales del módulo actual).
4. El alcance más externo (nombres integrados).

Si un nombre se declara `global`, todas las referencias y asignaciones van al ámbito de nombres globales. Para enlazar variables fuera del ámbito más interno, se usa `nonlocal`; de lo contrario, son de solo lectura y cualquier asignación creará una nueva variable local.

Ejemplo de ámbitos y espacios de nombre

Este es un ejemplo que muestra cómo hacer referencia a distintos ámbitos y cómo las declaraciones `global` y `nonlocal` afectan la asignación de variables:

```
def scope_test():
    def do_local():
        spam = "local spam"

    def do_nonlocal():
        nonlocal spam
        spam = "nonlocal spam"

    def do_global():
        global spam
        spam = "global spam"

    spam = "test spam"
    do_local()
    print("After local assignment:", spam)
    do_nonlocal()
    print("After nonlocal assignment:", spam)
    do_global()
    print("After global assignment:", spam)

scope_test()
print("In global scope:", spam)
```

El resultado del código ejemplo es:

```
After local assignment: test spam
After nonlocal assignment: nonlocal spam
After global assignment: nonlocal spam
In global scope: global spam
```

Notá cómo la asignación local (que es el comportamiento normal) no cambió la vinculación de *spam* de `scope_test`. La asignación `nonlocal` cambió la vinculación de *spam* de `scope_test`, y la asignación `global` cambió la vinculación a nivel de módulo. También podés ver que no había vinculación para *spam* antes de la asignación global.

2. Objetos, Instancias y Métodos

Un primer vistazo a las clases

Las clases introducen un poquito de sintaxis nueva, tres nuevos tipos de objetos y algo de semántica nueva.

Sintaxis de definición de clases

La forma más sencilla de definición de una clase se ve así:

```
class ClassName:
    <statement-1>
    .
    .
    .
    <statement-N>
```

Las definiciones de clases deben ejecutarse antes de que tengan efecto alguno. Cuando se ingresa una definición de clase, se crea un nuevo espacio de nombres, el cual se usa como ámbito local. Cuando finaliza normalmente, se crea un **objeto clase** que envuelve los contenidos de este espacio de nombres.

Objetos clase

Los objetos clase soportan dos tipos de operaciones: **hacer referencia a atributos** e **instanciación**.

Para hacer referencia a atributos se usa la sintaxis estándar: `objeto.nombre` . La instanciación de clases usa la notación de funciones, creando un objeto vacío o invocando automáticamente al método `__init__()` si está definido:

```
class Complex:
    def __init__(self, realpart, imagpart):
        self.r = realpart
        self.i = imagpart

x = Complex(3.0, -4.5)
print(x.r, x.i)  # (3.0, -4.5)
```

Objetos instancia y Objetos método

La única operación entendida por los objetos instancia es la referencia de atributos (datos y métodos). Los **atributos de datos** no necesitan ser declarados; se crean la primera vez que se les asigna algo. Los **métodos** son funciones que «pertenecen» a un objeto.

Lo que tienen de especial los métodos es que el objeto es pasado como el primer argumento de la función de forma implícita. La llamada `x.f()` es exactamente equivalente a `MyClass.f(x)` .

Variables de clase y de instancia

Las variables de instancia son para datos únicos de cada instancia, y las variables de clase son compartidas por todas las instancias. El uso incorrecto de variables de clase con objetos mutables (como listas) causa efectos inesperados:

```
class Dog:
    def __init__(self, name):
        self.name = name
        self.tricks = []    # Correcto: crea una lista vacía para cada perro

    def add_trick(self, trick):
        self.tricks.append(trick)
```

Algunas observaciones

- Si el mismo nombre de atributo aparece tanto en la instancia como en la clase, la búsqueda **prioriza la instancia**.
- En Python no hay nada que haga cumplir el ocultamiento de datos; todo se basa en **convención**. Los clientes pueden modificar o añadir atributos bajo su propio riesgo.
- No hay un atajo para hacer referencia a atributos desde dentro de un método, lo que aumenta la legibilidad al evitar confusiones entre variables locales y de instancia.

La convención "self"

A menudo, el primer argumento de un método se llama `self` . El nombre *self* no significa nada en especial para Python, pero es una convención universal. No seguirla hace que el código sea mucho menos legible para otros programadores.

Cualquier objeto función que es un atributo de clase define un método. Los métodos pueden llamar a otros métodos de la instancia usando el argumento `self` (ej. `self.add(x)`). Además, todo valor en Python es un objeto, y por lo tanto tiene una clase (su tipo), la cual se almacena como `objeto.__class__`.

3. Ejemplo POO Teletubbies

Ejercicio de POO: La Galaxia de los Teletubbies (Métodos Mágicos)

1. Objetivo

Diseñar e implementar en Python una clase que represente a un Teletubby, aplicando **atributos de clase**, **atributos de instancia (públicos y privados)**, **métodos de instancia** y los **métodos mágicos** `__str__` y `__repr__`.

2. Requisitos de la Clase (PersonajeTeletubby)

A. Atributo de clase

- Crea una variable de clase llamada `saludo_oficial` con el valor `"¡Tubby abrazo!"`.

B. Atributos de instancia (Constructor __init__)

- **Públicos:** `nombre`, `color`, `antena` y `objeto_favorito`.
- **Privados (__):** `__nivel_hambre` (50) y `__nivel_felicidad` (100).

C. Métodos de instancia

- `saludar()`: Presentación con nombre, color y antena.
- `jugar()`: Imprime que está jugando. Sube el `__nivel_hambre` (+20) y la `__nivel_felicidad` (+10).
- `comer_tubby_papilla()`: Baja el `__nivel_hambre` a 0 y sube la `__nivel_felicidad` (+20).
- `ver_estado()`: Devuelve una cadena con el nivel de felicidad y hambre.

D. Métodos Mágicos (Dunder Methods)

- `__str__(self)`: Debe devolver una cadena amigable y fácil de leer con un resumen del Teletubby (ej. "Teletubby Tinky Winky (Morado) - Feliz: 100").
- `__repr__(self)`: Debe devolver una cadena técnica que represente cómo se instanció el objeto (ej. `PersonajeTeletubby('Tinky Winky', 'Morado', 'Triángulo', 'Bolso')`).

3. Bloque Principal (main)

En el bloque `if __name__ == "__main__":`:

1. Instancia al menos **dos objetos** de la clase.
2. Haz que jueguen y coman utilizando sus métodos de instancia.
3. Prueba el método `__str__` simplemente haciendo un `print(objeto)`.
4. Prueba el método `__repr__` haciendo un `print(repr(objeto))` o imprimiendo una lista que contenga a los objetos.

> #### Teoría: Diferencias entre __str__ y __repr__
>
> En Python, ambos métodos sirven para convertir un objeto en una cadena de texto, pero tienen propósitos distintos:
>
> - `__str__` **(String):** Está pensado para el **usuario final**. Su objetivo es ser legible, descriptivo e informal. Se ejecuta automáticamente cuando usamos `print(objeto)` o `str(objeto)`.
> - `__repr__` **(Representation):** Está pensado para los **desarrolladores**. Su objetivo es no tener ambigüedades. Lo ideal es que devuelva una cadena de texto con código Python válido que permita recrear exactamente el mismo objeto. Se usa al invocar `repr(objeto)` o cuando imprimimos una lista/diccionario que contiene dichos objetos.
>
> *Regla de oro:* `__str__` es para leer; `__repr__` es para depurar (debugging).

4. Solución en Código Python

A continuación se muestra el código completo que resuelve el ejercicio:

```
class PersonajeTeletubby:
    # Atributo de clase
    saludo_oficial = "¡Tubby abrazo!"

    def __init__(self, nombre, color, antena, objeto_favorito):
        # Atributos públicos
        self.nombre = nombre
        self.color = color
        self.antena = antena
        self.objeto_favorito = objeto_favorito

        # Atributos privados
        self.__nivel_hambre = 50
        self.__nivel_felicidad = 100

    def saludar(self):
        print(f"¡Oh-oh! Soy {self.nombre}. Soy {self.color} y mi antena es un {self.antena}.")

    def jugar(self):
        print(f"{self.nombre} está jugando con su {self.objeto_favorito}.")
        self.__nivel_hambre += 20
        self.__nivel_felicidad += 10

    def comer_tubby_papilla(self):
        print(f"{self.nombre} se come un tazón de Tubby Papilla. ¡Slurp!")
        self.__nivel_hambre = 0
        self.__nivel_felicidad += 20

    def ver_estado(self):
        return f"Estado de {self.nombre} -> Felicidad: {self.__nivel_felicidad} | Hambre:
{self.__nivel_hambre}"

    # --- NUEVOS MÉTODOS MÁGICOS ---

    def __str__(self):
        # Orientado al usuario: formato amigable
        return f"Teletubby {self.nombre} (Color: {self.color}) - Estado actual: Hambre
{self.__nivel_hambre}"

    def __repr__(self):
        # Orientado al desarrollador: formato técnico para recrear el objeto
        return f"PersonajeTeletubby('{self.nombre}', '{self.color}', '{self.antena}',
'{self.objeto_favorito}')"


# Bloque Main para probar la clase
if __name__ == "__main__":
    # 1. Instanciamos los objetos
    tinky_winky = PersonajeTeletubby("Tinky Winky", "Morado", "Triángulo", "Bolso")
    po = PersonajeTeletubby("Po", "Rojo", "Círculo", "Patinete")

    # 2. Acciones normales
    tinky_winky.saludar()
    po.jugar()
    po.comer_tubby_papilla()

    print("\n--- PROBANDO MÉTODOS MÁGICOS ---")

    # 3. Probando __str__
    # Al hacer print() de un objeto, Python llama automáticamente a __str__
    print("Usando print() que llama a __str__:")
    print(tinky_winky)
    print(po)

    # 4. Probando __repr__
    # Forzamos la llamada a __repr__ usando la función repr()
    print("\nUsando repr() que llama a __repr__:")
    print(repr(tinky_winky))

    # Otra forma en la que actúa __repr__ es al imprimir colecciones (listas, diccionarios)
    print("\nImprimiendo una lista de Teletubbies (usa __repr__ automáticamente):")
```

```
    lista_teletubbies = [tinky_winky, po]
    print(lista_teletubbies)
```

4. Herencia

Herencia

Por supuesto, una característica del lenguaje no sería digna del nombre «clase» si no soportara herencia. La sintaxis para una definición de clase derivada se ve así:

```
class DerivedClassName(BaseClassName):
    <statement-1>
    .
    .
    .
    <statement-N>
```

El nombre `BaseClassName` debe estar definido en un ámbito que contenga a la definición de la clase derivada. En el lugar del nombre de la clase base se permiten otras expresiones arbitrarias. Esto puede ser útil, por ejemplo, cuando la clase base está definida en otro módulo:

```
class DerivedClassName(modname.BaseClassName):
```

La ejecución de una definición de clase derivada procede de la misma forma que una clase base. Cuando el objeto clase se construye, se tiene en cuenta a la clase base. Esto se usa para resolver referencias a atributos: si un atributo solicitado no se encuentra en la clase, la búsqueda continúa por la clase base. Esta regla se aplica recursivamente si la clase base misma deriva de alguna otra clase.

Instanciación y Redefinición de Métodos

No hay nada en especial en la instanciación de clases derivadas: `DerivedClassName()` crea una nueva instancia de la clase. Las referencias a métodos se resuelven buscando el atributo de clase correspondiente, descendiendo por la cadena de clases base si es necesario, y la referencia al método es válida si se entrega un objeto función.

Las clases derivadas pueden redefinir métodos de su clase base. Como los métodos no tienen privilegios especiales cuando llaman a otros métodos del mismo objeto, un método de la clase base que llame a otro método definido en la misma clase base puede terminar llamando a un método de la clase derivada que lo haya redefinido. (Para los programadores de C++: en Python todos los métodos son en efecto virtuales).

Un método redefinido en una clase derivada puede de hecho querer extender en vez de simplemente reemplazar al método de la clase base con el mismo nombre. Hay una manera simple de llamar al método de la clase base directamente: simplemente llamás a `BaseClassName.methodname(self, arguments)`. En ocasiones esto es útil para los clientes también (siempre que la clase base sea accesible en el ámbito global).

Funciones Integradas para Herencia

Python tiene dos funciones integradas que funcionan con herencia:

- Usar `isinstance()` para verificar el tipo de una instancia: `isinstance(obj, int)` será `True` sólo si `obj.__class__` es `int` o alguna clase derivada de `int`.

- Usar `issubclass()` para verificar la herencia de clases: `issubclass(bool, int)` es `True` ya que `bool` es una subclase de `int`. Sin embargo, `issubclass(float, int)` es `False` ya que `float` no es una subclase de `int`.

4.1. Herencia Múltiple y Polimorfismo

Para resolver qué método ejecutar cuando varias clases superpuestas comparten el mismo nombre, Python utiliza un algoritmo llamado **MRO** (*Method Resolution Order*).

En el siguiente ejemplo, la clase `Cyborg` hereda de `Humano` y de `Robot`. Ambas clases padre contienen el método `accion_principal()`.

Código de Ejemplo en Python

```
class Humano:
    def accion_principal(self):
        print("Humano: Pensando y sintiendo emociones.")

    def dormir(self):
        print("Humano: Durmiendo 8 horas...")


class Robot:
    def accion_principal(self):
        print("Robot: Procesando datos a gran velocidad.")

    def recargar(self):
        print("Robot: Batería cargándose...")


# HERENCIA MÚLTIPLE: Prioridad de izquierda a derecha -> primero Humano, luego Robot
class Cyborg(Humano, Robot):
    def __init__(self, nombre):
        self.nombre = nombre


# Bloque Main para probar el comportamiento
if __name__ == "__main__":
    terminator = Cyborg("T-800")

    # 1. RESOLUCIÓN AUTOMÁTICA
    # Como Humano va antes que Robot en class Cyborg(Humano, Robot),
    # Python ejecuta el método de Humano.
    print("--- Acción Principal del Cyborg ---")
    terminator.accion_principal()

    # 2. POLIMORFISMO Y HERENCIA DE MÉTODOS ÚNICOS
    # Puede usar métodos específicos de ambas clases padre sin conflicto
    terminator.dormir()
    terminator.recargar()

    # 3. ¿CÓMO ELEGIR EXPLÍCITAMENTE QUÉ MÉTODO USAR?
    # Si quieres forzar el uso del método de Robot (el segundo en la lista):
    print("\n--- Forzando la llamada al método de Robot ---")
    Robot.accion_principal(terminator)

    # 4. VER EL ORDEN DE RESOLUCIÓN (MRO)
    print("\n--- Orden de Resolución de Métodos (MRO) ---")
    for i, clase in enumerate(Cyborg.mro(), 1):
        print(f"{i}. {clase.__name__}")
```

¿Cómo soluciona Python qué método usar y cómo elegirlo?

- **Orden por defecto (MRO):** Al definir `class Cyborg(Humano, Robot):` , Python evalúa las clases padre de izquierda a derecha. Por eso, al invocar `terminator.accion_principal()` , se ejecuta automáticamente el método de la clase `Humano` .
- **Llamada explícita:** Si necesitas ejecutar específicamente la versión de la segunda clase padre (`Robot`), puedes hacerlo invocando el método directamente desde la clase y pasándole la instancia: `Robot.accion_principal(terminator)` .
- **Polimorfismo:** La subclase hereda todos los métodos únicos de sus padres (como `dormir()` de `Humano` y `recargar()` de `Robot`), respondiendo adecuadamente a la interfaz de cualquiera de ellos.

4.2. MRO en Python

MRO (Method Resolution Order) en Python

MRO son las siglas de **Method Resolution Order** (Orden de Resolución de Métodos).

Es el conjunto de reglas que utiliza Python para determinar **en qué orden debe buscar un método o un atributo** cuando este es invocado en un objeto, especialmente en escenarios con **herencia múltiple**.

¿Por qué existe y qué problema resuelve?

Cuando un objeto llama a un método:

```
objeto.mi_metodo()
```

Python tiene que decidir qué implementación de `mi_metodo()` debe ejecutar. Si la clase de `objeto` no define ese método pero hereda de dos clases padre que **ambas tienen un método con el mismo nombre**, surge un conflicto de ambigüedad (conocido en teoría de programación como el *Problema del Diamante*).

El MRO es el algoritmo de Python que resuelve esta ambigüedad creando una **lista lineal estricta y predecible** con el orden de búsqueda.

Las 3 reglas clave del MRO

1. **De abajo hacia arriba:** Busca primero en la subclase (el objeto actual) antes que en cualquiera de sus clases padre.
2. **De izquierda a derecha:** Si una clase hereda de varias (`class C(A, B):`), buscará en A antes que en B.
3. **Sin duplicados / Mantiene la jerarquía:** Utiliza el algoritmo *C3 Linearization*, garantizando que una clase padre siempre se busque después de sus subclases y sin repeticiones. La clase base universal `object` va al final.

¿Cómo consultar el MRO de una clase?

Puedes ver la ruta exacta de búsqueda que seguirá Python de tres formas:

```
# 1. Usando el método .mro()
print(MiClase.mro())

# 2. Consultando el atributo especial .__mro__
print(MiClase.__mro__)

# 3. Usando la función help() en la consola interactiva
help(MiClase)
```

Ejemplo de salida

El clase `Cyborg` , que hereda de `Humano` y `Robot` ,:

```
class Cyborg(Humano, Robot):
```

Si ejecutas `Cyborg.mro()`, obtendrás una lista como esta:

```
[Cyborg, Humano, Robot, object]
```

Esto le indica a Python: *"Busca primero en `Cyborg`. Si no está, busca en `Humano`. Si no está, busca en `Robot`. Si no está, busca en `object`. Si no está en ninguno, lanza un `AttributeError`".*

Ejemplo de Código: Humano, Robot y Cyborg

A continuación se muestra el código funcional para probar cómo se resuelve el MRO y cómo usar `super()` para acceder a la jerarquía de llamadas:

```
class Humano:
    def accion_principal(self):
        return "Respirar y pensar lógicamente."

class Robot:
    def accion_principal(self):
        return "Procesar datos mecánicamente."

# La clase Cyborg hereda de Humano y Robot (Regla de izquierda a derecha)
class Cyborg(Humano, Robot):
    def accion_principal(self):
        # super() sigue el MRO, por lo que llamará al método de Humano
        accion_base = super().accion_principal()
        return f"Cyborg operando: {accion_base} (con mejoras de hardware)"

# 1. Instanciación y prueba del método
c = Cyborg()
print("Resultado del método:")
print(c.accion_principal())
print("-" * 40)

# 2. Consultar el MRO
print("MRO usando .mro():")
print(Cyborg.mro())
print("\nMRO usando .__mro__:")
print(Cyborg.__mro__)
```

Salida Esperada

Al ejecutar el código, Python devuelve la cadena de resolución exacta y el resultado de invocar `super()` basándose en esa misma cadena:

```
Resultado del método:
Cyborg operando: Respirar y pensar lógicamente. (con mejoras de hardware)
----------------------------------------
MRO usando .mro():
[<class '__main__.Cyborg'>, <class '__main__.Humano'>, <class '__main__.Robot'>, <class 'object'>]

MRO usando .__mro__:
(<class '__main__.Cyborg'>, <class '__main__.Humano'>, <class '__main__.Robot'>, <class 'object'>)
```

Esto le indica a Python: "Busca primero en *Cyborg*. Si no está, busca en *Humano*. Si no está, busca en *Robot*. Si no está, busca en *object*. Si no está en ninguno, lanza un `AttributeError`".

4.3. Herencia múltiple cooperativa

Código completo para probar el MRO con argumentos

Aquí tienes un script funcional y listo para ejecutar. Este ejemplo implementa la **herencia múltiple cooperativa** utilizando `**kwargs`, lo que permite que `super()` propague la llamada a través de toda la cadena del MRO (Cyborg -> Humano -> Robot) sin perder los argumentos.

```
class Humano:
    def ejecutar_tarea(self, tarea, prioridad, **kwargs):
        print(f"[Humano] Intentando '{tarea}' con prioridad {prioridad}.")
        # Pasa la llamada al siguiente en el MRO (en este caso, Robot)
        if hasattr(super(), 'ejecutar_tarea'):
            super().ejecutar_tarea(tarea, prioridad, **kwargs)

class Robot:
    def ejecutar_tarea(self, tarea, prioridad, **kwargs):
        print(f"[Robot] Procesando '{tarea}' a nivel {prioridad}.")
        # object (la clase base final) no tiene ejecutar_tarea, así que aquí termina.

class Cyborg(Humano, Robot):
    def ejecutar_tarea(self, tarea, prioridad, **kwargs):
        print(f"[Cyborg] Iniciando directiva principal...")

        # Llamada MRO en cascada
        print("\n--- Ejecución vía super() (MRO) ---")
        super().ejecutar_tarea(tarea, prioridad, **kwargs)

        # Llamadas explícitas saltándose el MRO cooperativo
        print("\n--- Ejecuciones Explícitas Aisladas ---")
        Humano.ejecutar_tarea(self, tarea, prioridad) # Nota: esto desencadenará a Robot también por el
super() interno de Humano
        Robot.ejecutar_tarea(self, tarea, prioridad)

# 1. Instanciamos el objeto
terminator = Cyborg()

# 2. Ejecutamos el método con argumentos
terminator.ejecutar_tarea("Proteger a John Connor", 1)

# 3. Comprobamos el orden del MRO
print("\n--- Cadena MRO de la clase Cyborg ---")
print([cls.__name__ for cls in Cyborg.mro()])
```

Salida esperada al ejecutarlo:

```
[Cyborg] Iniciando directiva principal...

--- Ejecución vía super() (MRO) ---
[Humano] Intentando 'Proteger a John Connor' con prioridad 1.
[Robot] Procesando 'Proteger a John Connor' a nivel 1.

--- Ejecuciones Explícitas Aisladas ---
[Humano] Intentando 'Proteger a John Connor' con prioridad 1.
[Robot] Procesando 'Proteger a John Connor' a nivel 1.
[Robot] Procesando 'Proteger a John Connor' a nivel 1.

--- Cadena MRO de la clase Cyborg ---
['Cyborg', 'Humano', 'Robot', 'object']
```

5. Variable privadas y Tipado dinámico

Variables privadas

Las variables «privadas» de instancia, que no pueden accederse excepto desde dentro de un objeto, no existen en Python. Sin embargo, hay una convención que se sigue en la mayoría del código Python: un nombre prefijado con un guion bajo (por ejemplo, `_spam`) debería tratarse como una parte no pública de la API (más allá de que sea una función, un método, o un dato). Debería considerarse un detalle de implementación y que está sujeto a cambios sin aviso.

Ya que hay un caso de uso válido para los identificadores privados de clase (a saber: colisión de nombres con nombres definidos en las subclases), hay un soporte limitado para este mecanismo. Cualquier identificador con la forma `__spam` (al menos dos guiones bajos al principio, como mucho un guion bajo al final) es textualmente reemplazado por `_nombredeclase__spam`, donde *nombredeclase* es el nombre de clase actual al que se le sacan guiones bajos del comienzo (si los tuviera). Se modifica el nombre del identificador sin importar su posición sintáctica, siempre y cuando ocurra dentro de la definición de una clase.

Ejemplo de Modificación de Nombres (Name Mangling)

La modificación de nombres es útil para dejar que las subclases sobreescriban los métodos sin romper las llamadas a los métodos desde la misma clase. Por ejemplo:

```
class Mapping:
    def __init__(self, iterable):
        self.items_list = []
        self.__update(iterable)

    def update(self, iterable):
        for item in iterable:
            self.items_list.append(item)

    __update = update   # private copy of original update() method

class MappingSubclass(Mapping):
    def update(self, keys, values):
        # provides new signature for update()
        # but does not break __init__()
        for item in zip(keys, values):
            self.items_list.append(item)
```

El ejemplo de arriba funcionaría incluso si `MappingSubclass` introdujera un identificador `__update` ya que se reemplaza con `_Mapping__update` en la clase `Mapping` y `_MappingSubclass__update` en la clase `MappingSubclass` respectivamente.

Observaciones sobre la Privacidad

- Las reglas de modificación de nombres están diseñadas principalmente para **evitar accidentes**; es posible acceder o modificar una variable que es considerada como privada (útil en el depurador).
- El código pasado a `exec` o `eval()` no considera que el nombre de clase de la clase que invoca sea la clase actual. La misma restricción aplica a `getattr()`, `setattr()` y `delattr()`, así como cuando se referencia a `__dict__` directamente.

Detalles y Cuestiones Varias

A veces es útil tener un tipo de datos similar al «registro» de Pascal o la «estructura» de C, que sirva para juntar algunos pocos ítems con nombre. El enfoque idiomático es utilizar **dataclasses** con este propósito:

```
from dataclasses import dataclass

@dataclass
class Employee:
    name: str
    dept: str
    salary: int

john = Employee('john', 'computer lab', 1000)
print(john.dept)    # 'computer lab'
print(john.salary)  # 1000
```

Tipado Dinámico y Objetos Método

Un fragmento de código en Python que espera un tipo de dato abstracto en particular a menudo puede recibir una clase que emule los métodos de ese tipo de dato en su lugar. Por ejemplo, si tienes una función que formatea algunos datos de un objeto de archivo, puedes definir una clase con los métodos `read()` y `readline()` que obtienen los datos de un búfer de cadena en su lugar, y pasarla como argumento.

Los objetos de método de instancia también tienen atributos: `m.__self__` es el objeto de instancia con el método `m()`, y `m.__func__` es el objeto de función correspondiente al método.

6. Iteradores y Generadores

Iteradores

Es probable que hayas notado que la mayoría de los objetos contenedores pueden ser recorridos usando una sentencia `for`:

```
for element in [1, 2, 3]:
    print(element)

for element in (1, 2, 3):
    print(element)

for key in {'one':1, 'two':2}:
    print(key)

for char in "123":
    print(char)

for line in open("myfile.txt"):
    print(line, end='')
```

Este estilo de acceso es limpio, conciso y conveniente. El uso de iteradores está impregnado y unifica a Python. En bambalinas, la sentencia `for` llama a `iter()` en el objeto contenedor. La función retorna un objeto iterador que define el método `__next__()` que accede elementos en el contenedor de a uno por vez. Cuando no hay más elementos, `__next__()` lanza una excepción **StopIteration** que le avisa al bucle del for que hay que terminar.

Creando un Iterador Personalizado

Habiendo visto la mecánica del protocolo de iteración, es fácil agregar comportamiento de iterador a tus clases definiendo los métodos `__iter__()` y `__next__()` :

```
class Reverse:
    """Iterator for looping over a sequence backwards."""
    def __init__(self, data):
        self.data = data
        self.index = len(data)

    def __iter__(self):
        return self

    def __next__(self):
        if self.index == 0:
            raise StopIteration
        self.index = self.index - 1
        return self.data[self.index]

rev = Reverse('spam')
for char in rev:
    print(char)
# m a p s
```

Generadores

Los generadores son una herramienta simple y poderosa para crear iteradores. Están escritas como funciones regulares pero usan la palabra clave `yield` siempre que quieran retornar datos. Cada vez que se llama a `next()`, el generador se reanuda donde lo dejó (recuerda todos los valores de datos y qué instrucción se ejecutó por última vez).

```
def reverse(data):
    for index in range(len(data)-1, -1, -1):
        yield data[index]

for char in reverse('golf'):
    print(char)
# f l o g
```

Ventajas de los Generadores

- Los métodos `__iter__()` y `__next__()` son creados automáticamente.
- Las variables locales y el estado de la ejecución son guardados automáticamente entre llamadas, haciendo el código más limpio que usar variables de instancia (como `self.indice`).
- Lanzan **StopIteration** de manera automática cuando terminan.

Expresiones generadoras

Algunos generadores simples pueden ser escritos de manera concisa como expresiones usando una sintaxis similar a las comprensiones de listas pero con **paréntesis en lugar de corchetes**. Estas expresiones están hechas para situaciones donde el generador es utilizado de inmediato por la función que lo encierra. Las expresiones generadoras son más compactas pero menos versátiles que las definiciones completas de generadores y tienden a ser más amigables con la memoria que sus comprensiones de listas equivalentes.

```python
# Suma de cuadrados
sum(i*i for i in range(10))  # 285

# Producto punto
xvec = [10, 20, 30]
yvec = [7, 5, 3]
sum(x*y for x,y in zip(xvec, yvec))  # 260

# Otros usos útiles
unique_words = set(word for line in page for word in line.split())
valedictorian = max((student.gpa, student.name) for student in graduates)

data = 'golf'
list(data[i] for i in range(len(data)-1, -1, -1))  # ['f', 'l', 'o', 'g']
```